Pricing via Artificial Intelligence: The Impact of Neural Network Architecture on Algorithmic Collusion

Weipeng Zhang *

June 2023

Abstract

Classic artificial intelligence (Q-learning) algorithms have been capable of consistently learning supra-competitive pricing strategies in infinitely repeated Nash-Bertrand pricing games without human communication. Such algorithms have been able to converge due to the temporal correlation of consecutive states and actions in the learning process, which restores stationarity in an otherwise highly non-stationary setting. It is difficult for more realistic AI algorithms to converge, as the necessary training processes breaks the aforementioned temporal correlation, rendering the algorithms ineffective in learning reward-punishment strategies that result in collusive market outcomes. We adapt several widely used neural network architectures to the framework of model-free reinforcement learning and experimentally explore how the structure of AI algorithms affects market outcomes in a workhorse oligopolistic model of repeated price competition. While it is possible to train advance AI algorithms to always best respond in environments where the rival exercises a fixed strategy, it is unlikely that such algorithms can learn to coordinate in setting supra-competitive prices due to the non-stationarity of multi-agent learning processes, suggesting that algorithmic collusion may not be an immediate concern for antitrust authorities.

JEL: C72, C73, D43, K21, L1, L13, L4, L51

Keywords: Repeated games, algorithmic collusion, experience replay, deep reinforcement learning, deep Q-learning, deep recurrent Q-learning

*Department of Economics, Georgetown University. Email: wz167@georgetown.edu

Electronic copy available at: https://ssrn.com/abstract=4489688

1 Introduction

Given the wealth of data available to infer consumer preferences and advances in artificial intelligence, firms have increasingly delegated business decisions to algorithms. Such algorithms may use learning processes to engage in experimentation and adapt to evolving environments, becoming capable of developing optimal pricing strategies with little to no human guidance. While it is illegal for firms to explicitly collude in price setting, policy makers and antitrust authorities have argued that such algorithms can potentially learn to collude tacitly without any human communication, thereby undermining competition in the markets for which they operate. This paper investigates how the design of artificial intelligence algorithms can lead to competitive or supra-competitive pricing outcomes when competing in a Nash-Bertrand pricing game.

2 Related Literature

Recent empirical studies have shown the significant extent to which software algorithms have been adopted by firms in setting prices for their goods. Chen et al. (2016) detects that roughly one third of sellers on Amazon Marketplace have implemented algorithmic pricing in a sample of best-seller products. Brown & MacKay (2021) analyzed pricing in the over-the-counter drugs market and showed that firms with superior pricing technology exhibited meaningful increases to markups. Assad et al. (2020) presents evidence that the adoption of algorithmic pricing software in German retail gasoline markets increases margins by 9%. More broadly, artificial intelligence algorithms have been used to generate recommendations for goods and services that affect consumer welfare. Gomez-Uribe et al. (2016) highlights that four out of five movies watched on Netflix were algorithm recommended. Chen et al. (2021) shows that products sold by Amazon receive substantially more "Frequently Bought Together" recommendations across product categories and popularity deciles and results in a loss of efficiency. Hence, the increasing use of artificial intelligence warrants additional investigation into the structure of such algorithms and how they impact market outcomes.

There are numerous theoretical studies analyzing how artificial intelligence might facilitate collusion and lead to supra-competitive market outcomes. Harrington Jr. (2018) argues that current regulatory approaches are insufficient when algorithms can learn collusive strategies tacitly, without communication nor specifically being instructed to cooperate and thus warrants serious discussion. One stream of works including Miklós-Thal & Tucker (2019) and O'Connor & Wilson (2021) examines the role algorithms have in improving predictions and allowing colluding firms to better tailor prices to demand conditions. Another series of papers explore the implications commitment to pricing rules has on producing collusive outcomes. Salcedo (2015) considers the case in which firms commit to pricing algorithms in the short run with the later possibility of revision. Such algorithms not only facilitate collusion but also makes collusion inevitable, i.e. in any equilibrium, firms achieve joint profits close to those of a monopolist. Harrington Jr. (2021) explores pricing algorithms that allow firms to condition price on high-frequency demand information can lead to anticompetitive effects that increase in the substitutability of goods. Brown & MacKay (2021) show that in

Electronic copy available at: https://ssrn.com/abstract=4489688

competitive Markov perfect equilibrium, simple pricing algorithms can generate price dispersion, increase price levels, and exacerbate the price effects of mergers. Leisten (2021) produces a unified theory in distinguishing the prediction-enhancing and commitment-enhancing features of algorithms: either one or the other is enough to sustain supra-competitive prices.

Due to the paucity of rich market data and the difficulty of detecting algorithmic collusion empirically, research on this topic has primarily focused on experimental approaches to address what types of algorithms antitrust authorities should consider. Given the black box nature of algorithms being used by firms, one possibility is that policymakers can audit the code of such algorithms to determine if they are consistent with the designs deemed to be capable of collusive behavior as suggested in Assad et al. (2020). This literature has constructed simple AI pricing agents that interact in computer-simulated pricing games for which the outcomes have been analyzed. Calvano et al. (2020), the first paper to argue that autonomous pricing algorithms can collude, examines model-free reinforcement learning (Q-learning) algorithms and their propensity to collude and learn strategies that punish defections. Such algorithms, to be described in Section 4, maximize the sum of discounted rewards in problems of repeated choice and serve as a natural starting point for the study of algorithmic collusion. Based on their results, they argue that if simple algorithms can learn to collude, more complex algorithms including those that employ "deep learning" would lead to higher risk of tacit collusion. In contention to this view, we show that more complex algorithms such as deep Q-learning, depending on their design, may actually foster more competition than naive Q-learning. We examine several different architectures of such algorithms and discuss their propensity for collusion. Our work suggests that the inability to collude is attributed to the lack of convergence guarantees and non-stationarity of the learning processes that cannot be mitigated easily in more complex algorithms. Klein (2020) finds that when the environment is one of sequential competition in which firms alternate in setting prices each period, two Q-learning algorithms converge to either a constant price or to a pattern resembling an Edgeworth cycle, with profits on average exceeding Nash equilibrium levels. Hansen et al. (2021) show that when competing sellers use misspecified machine learning algorithms, they can overestimate their price sensitivity and consequently set supra-competitive prices. Asker et al. (2022) shows if two firms adopt symmetric "asynchronous learning" algorithms, known in the reinforcement learning literature as Q-learning, then market prices converge to rest points that are vastly above Nash equilibrium prices. Under their setting, if the algorithms firms adopt alternatively employ "synchronous learning", in which profits for all possible actions are computed for a fixed price of the opponent, competition is restored and market prices quickly converge to Nash equilibrium prices. However, the latter case unrealistically assumes that firms are unboundedly rational and have full knowledge of the market demand function and how their price taken together with competitor prices map to payoffs. In this paper, we describe a different kind of algorithmic design that is able to destabilize collusion and generate competitive prices under the assumption that firms operate and undergo learning in a model-free setting.

Our work is also related to the extensive literature on multi-agent reinforcement learning. In the con-

Electronic copy available at: https://ssrn.com/abstract=4489688

text of single-agent Markov decision problems, there are theoretical convergence guarantees for both the Q-learning algorithm and deep Q-learning algorithm as presented in Watkins & Dayan (1992) and Fan et al. (2020), respectively. However, there are no sufficiency conditions for general-sum games that guarantee analogous convergence results in the multi-agent setting, where the environment of each agent becomes non-stationary as it includes the actions of other agents who also learn and experiment. We refer interested readers to Zhang et al. (2021), which presents a comprehensive survey of the latest developments in the theory and algorithms of multi-agent reinforcement learning. The lack of theoretical convergence results in the literature is only theoretically agnostic to the optimality or quality of policies derived from such algorithms but does not preclude their empirical convergence, which has been achieved in a large number of works including Vinyals et al. (2019) which trained multi-agent reinforcement learning algorithm using deep neural networks to achieve a performance rating above 99.8% of players in StarCraft II, a real time strategy computer game that has been considered one of the most challenging games to solve due to its large state-action space. We continue down this avenue to construct reinforcement learning agents that utilize rich function approximation techniques such as long short-term memory (LSTM), let them interact repeatedly in a computer-simulated economic environment, and explore whether such learning algorithms converge. We argue that such algorithms are representative of those that can be naturally employed in practice.

The remainder of this paper is organized as follows. In Section 3, we introduce the Bertrand-Nash pricing game that serves as the environment in which we experiment. In Section 4, we provide a brief overview of Q-learning, the workhorse algorithm used to study algorithmic collusion, and how it generalizes to the multi-agent setting. In Section 5 and Section 6, we introduce and motivate the use of deep neural networks as functional approximations in conjunction with Q-learning and present our experimental results. Section 7 performs robustness checks and Section 8 concludes.

3 Experimental Design

In this section, we describe the demand and supply system of a flexible workhorse Bertrand oligopoly model, the explicit algorithms with which the firms will employ to learn, and other specifications of the numerical simulations. We omit the subscripts t and present the assumptions on the infinitely repeated stage game.

3.1 Economic Environment

The demand model is based on a discrete choice model with differentiated products. Let J be the number of all products available in a certain market and let there be a measure one of consumers with utility

$$U_j = \frac{a_j - p_j}{\mu} + \epsilon_j,$$

where a_j is the product quality or the index vertical differentiation, and μ is the index of horizontal differentiation, and p_j is the price of product j. Product 0 is the outside good. We assume that ϵ_j for $j = 1, ..., J$

Electronic copy available at: https://ssrn.com/abstract=4489688

are i.i.d. Type I Extreme Value. The consumer will choose the good j that maximizes U_j and thus can write the demand for product j as

$$q_j = \frac{\exp\left(\frac{a_j - p_j}{\mu}\right)}{\sum_{r=1}^{J} \exp\left(\frac{a_r - p_r}{\mu}\right) + \exp\left(\frac{a_0}{\mu}\right)}.$$

There are N firms in the market and each firm $i \in \{1, ..., N\}$ sells a subset G_i of the J products. Let q_j denote the number sold of product j, mc_j denote the marginal cost of producing the product j, and p denote the vector of J product prices. The profit of firm i in period t is given by

$$\pi_i(p) = \sum_{j \in G_i} (p_j - mc_j) q_j(p).$$

Hence, the Nash equilibrium $\boldsymbol{p}^*$ of the stage game satisfies the following first order conditions for any $j \in G_i, i = 1, ..., N$:

$$\begin{aligned} \frac{\partial \pi_i}{\partial p_j}(\boldsymbol{p}^*) = 0 &\iff q_j(\boldsymbol{p}^*) + \sum_{r \in G_i} (p_r^* - mc_r) \frac{\partial q_r}{\partial p_j}(\boldsymbol{p}^*) = 0 \\ &\iff \boldsymbol{p}^* - \boldsymbol{mc} = \Delta(\boldsymbol{p}^*)^{-1} s(\boldsymbol{p}^*), \end{aligned}$$

where the elements of matrix $\Delta(\boldsymbol{p})$ of size $J \times J$ are $\Delta(\boldsymbol{p})_{jr} = -\frac{\partial q_r}{\partial p_j}$ if products r and j are produced by the same firm and 0 otherwise.

Note that this baseline model is deterministic but one can allow for a more flexible model where each of the model parameters could be subject to random shocks. Calvano et al. (2020) explores the case in which the inverse index of aggregate demand a_0 is stochastic and the case of stochastic entry and exit. We focus all subsequent analysis on the deterministic baseline model.

4 Q-Learning

We discuss classic tabular Q-Learning which was originally presented in Watkins (1989) for approximating the fixed point to Bellman's equation of single agent Markov decision processes. In each period $t = 0, 1, 2, ...$, the agent observes a state variable $s_t \in S$ from a finite set of possible states and chooses an action $a_t \in A(s_t)$ from a finite constraint set. The agent then incurs a period payoff π_t and the next state s_{t+1} is realized according to a stationary probability distribution $F(\pi_t, s_{t+1} | s_t, a_t)$. The agent maximizes his expected sum of discounted payoffs:

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \delta^t \pi_t\right],$$

where $0 < \delta < 1$ is the discount factor. Hence, the agent's Bellman's equation can be formulated as follows:

$$V(s) = \max_{a \in A} \mathbb{E}[\pi | s, a] + \delta \mathbb{E}[V(s') | s, a],$$

where s' is the next state realized after taking action a when the state is s.

Electronic copy available at: https://ssrn.com/abstract=4489688

To solve this problem, we consider the Q-function $Q(s,a)$ which gives the cumulative discounted payoff of taking action a in state s in the current period and taking the optimal action forever after:

$$Q(s,a) = \mathbb{E}[\pi|s,a] + \delta\mathbb{E}[\max_{a'\in A} Q(s',a')|s,a],$$

which is related to the value function by the identity $V(s) \equiv \max_{a\in A} Q(s,a)$. The Q-function is the object of interest for which the Q-learning algorithm seeks to estimate without imposing any assumptions on the underlying model, i.e. $F(\pi, s'|s,a)$. Once known, we are able to calculate the optimal action for any given state. The Q-learning method proceeds as follows. First, an arbitrary matrix Q_0 of size $|S|\times|A|$ is initialized. Upon choosing action a_t and s_t, the element of the matrix $Q_t(s,a)$ for $s = s_t$ and $a = a_t$ is updated according to

$$Q_{t+1}(s,a) = (1-\alpha)Q_t(s,a) + \alpha\left(\pi_t + \delta \max_{a\in A} Q_t(s',a)\right),$$

where $\alpha \in [0,1]$ is the learning rate. All other elements of the matrix remain unchanged. Under general conditions detailed in Watkins & Dayan (1992), Q-learning algorithms converge to the optimal policy with no prior knowledge of the underlying model $F(\pi, s'|s,a)$. If the learning algorithm is to uncover the optimal policy then all actions must be tried in all states. Hence, the algorithm faces the trade-off of exploration versus exploitation. The first indicates that the algorithm chooses sub-optimal actions given accumulated knowledge in order to gain new information about the quality of different actions for a given state in order to improve future decisions and the latter indicates that the algorithm chooses to exploit its stock of acquired knowledge.

An exogenous exploration policy of ε-greedy has typically been adopted in empirical exercises. That is, with probability ε_t the action at t is selected randomly among the feasible set, where all actions can be chosen with equal probability and with probability $1-\varepsilon_t$, the action is chosen greedily, i.e. the a which maximizes $Q_t(s_t,a)$ is chosen. The probability $\varepsilon_t = \exp(-\beta t)$ is allowed to decline over time as the algorithm accumulates more information about its environment, where $\beta > 0$ is a parameter. In the next subsection, we discuss how the Q-learning algorithm is adapted to a setting where multiple firms learn simultaneously.

4.1 Multi-Agent Q-Learning

In our analysis, we adopt a multi-agent framework in which firms simultaneously deploy the Q-learning algorithm. Specifically, for the infinitely repeated game detailed in Section 3, firm i seeks to maximize its expected discounted sum of payoffs given by

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \delta^t \pi_{i,t}\right],$$

where $0 < \delta < 1$ is the common discount factor across all firms. We assume each firm only sells one product so N is both the number of firms and products in the market. We define the state space S to be the set of all prices in the last k periods and hence the size of the state space is $|S| = m^{Nk}$. In each period, each

Electronic copy available at: https://ssrn.com/abstract=4489688

firm observes a common state $s_t \in S$ in a finite set of possible state and chooses a price for each good in the action space A. Similar to Calvano et al. (2020), we set the feasible set of prices for each good to be m equally spaced points in the interval $[(1-\zeta)p^N, (1+\zeta)p^M]$, where p^N is the Bertrand-Nash equilibrium price, p^M is the monopoly price, and $\zeta > 0$ is a parameter.

For each firm, we initialize the Q-function $Q_{i,0}(s,a)$ randomly. An initial state state s_0 is drawn randomly at the beginning of training. Each firm i chooses an action $a_{i,t}(s)$ that satisfies $a_{i,t}(s) = \text{argmax}[Q_{i,t}(a,s)]$. The state s_{t+1} is determined and the Q-function is updated. As aforementioned, for general-sum game theoretical problems, there are no general convergence guarantees in that we do not know whether the algorithms converge at all or, if they do, whether they converge to a Nash equilibrium. This is because in multi-agent Q-learning, agents' actions enter into future periods' state variables so consequently the environment becomes non-stationary as competitor's policies are changing over periods given they are also learning and experimenting. Following Calvano et al. (2020) and Asker et al. (2022), we verify the convergence experimentally. Moreover, instead of specifying a convergence criteria like Calvano et al. (2020), our learning process continues for a given number of periods after which the long run behavior of the agents is examined. In the next section, we develop several Q-learning algorithms that leverage different types of training procedures and neural network architectures and compare their performance to seminal works in the algorithmic collusion literature.

5 Deep Q-Networks

Calvano et al. (2020) argues that if simple algorithms can learn to collude, more complex algorithms including those that employ deep learning would lead to higher risk of tacit collusion. This calls attention to investigating how different algorithmic designs facilitate collusion and which structures of algorithms should policymakers be concerned about. We show that more complex algorithms such as deep Q-learning, depending on their design, may actually foster more competition than naive Q-learning. We examine several different architectures of such algorithms and discuss their propensity for collusion.

Deep neural networks (DNNs) are nonlinear function approximators that bear resemblance to the biological neural networks that constitute animal brains. They consist of a collection of neurons known as activation units that are connected through various layers. DNNs are widely used due to the universal approximation theorem that states that any function $y = f(x)$ can be approximated by a DNN with arbitrary precision. In this section, we examine DNNs of the form $f : \mathbb{R}^{nk} \to \mathbb{R}^m$ that takes as input an element of a state space and outputs a value for every element in the action space, i.e.

$$f(s) = W_2 \boldsymbol{\sigma}(W_1 s + v_1) + v_2,$$

where $W_1 \in \mathbb{R}^{h \times nk}$, $v_1 \in \mathbb{R}^h$, $W_2 \in \mathbb{R}^{m \times h}$, and $v_2 \in \mathbb{R}^m$ are the network parameters or "weights" θ. $\boldsymbol{\sigma}(\cdot)$ is a nonlinear activation function applied element-wise to the input argument. In this section we use the rectified linear unit (ReLu) $\sigma(x) = \max\{x, 0\}$ as the activation function. We examine a setting in which two firms

Electronic copy available at: https://ssrn.com/abstract=4489688

use a fully-connected neural network with a single hidden layer of size $h = 32$. The goal is to approximate the true Q-function $Q^*(s, a)$ with the DNN $Q(s, a|\theta)$, where θ are the weights of the neural network. We call $Q(s, a|\theta)$ the Q-network, which was introduced in the pioneering work of Mnih et al. (2015) which trained Q-networks to achieve superhuman performance in classic Atari games such as *pong* and *Video Pinball*.

There are several advantages that deep Q-learning has over standard Q-learning described in the previous section. First, the experience attained from one state-action pair improves the estimate of neighboring state-action pairs by generalization since parameters are updated instead of the individual Q-values of state-action pairs. Second, when the state and action spaces are large, the size of matrix representations of action-values increases exponentially in the number of state-action pairs. A smooth approximation of the Q-matrix may be used to bypass the curse of dimensionality and expedite the learning process. We now introduce the algorithm then comment on differences between deep Q-learning and standard Q-learning used in Calvano et al. (2020).

Algorithm 1: Multi-agent deep Q-learning with random experience replay

```
For each agent i = 1, ..., N:
Initialize a replay memory D^i with capacity M;
Initialize a Q-network Q^i with random weights θ^i;
Initialize a target Q-network Q̂^i with random weights θ̂^i;
Initialize a random state s_1;
for episode = 1,..., M do
    for period t = 1,...,T do
        For each agent i, with probability 1 − ε_t select a greedy action
        a_t^i = argmax_a Q^i(s_t, a; θ^i); otherwise select a random action a_t^i
        Execute actions (a_t^1, ..., a_t^n) together and each agent observes all rewards r_{i,t}
        Set state s_{t+1} according to actions taken
        Store transition {s_t, (a_t^1, ..., a_t^n), s_{t+1}, (r_{1,t}, ..., r_{n,t})} in every D^i
        Sample for each agent i a random mini-batch of transitions independently from D^i
        For each agent i, perform a gradient descent step on individual loss function with respect to
          weights θ^i, i.e. θ ← θ − α∇_θ loss
    end
    Every C episodes, update target network for each agent, i.e. θ̂^i ← θ^i
end
```

The deep Q-learning algorithm (Algorithm 1) starts by initializing two separate neural networks, one for each firm in the simulated market. The deep Q-learning algorithm possesses two salient features that are absent in the standard Q-learning algorithm: experience replay and a target network. The first technique of experience replay addresses the issue of temporal correlation between consecutive values of each agent's states and actions. If weights of the network θ are updated only with consecutive samples

Electronic copy available at: https://ssrn.com/abstract=4489688

of correlated experiences as they are collected sequentially from the environment, the learning becomes inefficient. Hence, we introduce a replay buffer for each agent in which their experiences in each period t, (s_t, a_t, r_t, s_{t+1}), is stored. When the network weights are updated in each period, a randomly selected subset of experiences in the replay buffer is used in minimizing a loss function. In other words, we first approximate $r + \delta \max_{a' in A'} Q^*(s', a')$ with $r + \delta \max_{a' \in A'} Q(s', a'|\theta^-)$, where θ^- are the weights of the target network and then formulate the loss function as follows for each experience we have sampled from the replay buffer:

$$\mathcal{L}(\theta) = Q(s, a|\theta) - \left[r + \delta \max_{a' \in A} Q(s', a'|\theta^-) \right].$$

The gradient with respect to θ is taken on the loss function which is given by the sum of all $\mathcal{L}(\theta)$ for each experience in the sampled mini-batch. The network weights are then updated. The benefits of using experience replay is twofold. There is greater data efficiency and less interactions with the environment are needed for convergence and the learning is more stable: when θ is updated according to one particular experience, the Q-function changes not only at the prescribed state-action pair but rather globally across the entire state-action space. Hence, by updating θ with only one particular experience tuple, the discrepancy between the true Q-function and the neural network approximation may be amplified at distant state-action pairs. By incorporating additional experiences in the loss function used to compute the gradient, such a discrepancy is reduced. The second technique of using a target network allows the learning algorithm to bypass the problem of chasing a moving target. That is, the weights θ^- are kept fixed for C episodes before being updated to the latest network weights θ in order to mitigate divergence in training.

5.1 Results

In this section, we report our results with an economic environment that consists of a symmetric duopoly. The baseline environment follows Calvano et al. (2020) and has $N = 2$ firms, each offering a good with $mc_i = 1$, $a_i = 2$, $a_0 = 0$, $\mu = 0.25$. Hence the Bertrand-Nash price p^N and monopoly price p^M are approximately 1.47 and 1.93, respectively. Firms have a discount rate of $\delta = 0.95$ and take the past $k = 1$ periods of prices as the state variable. The two deep Q-networks are trained with an exploration rate of $\beta = 1\text{e}{-5}$, a learning rate of $\alpha = 2.5\text{e}{-3}$. The replay memory for each agent is of size $M = 2000$, the batch size is 128, and the target network is updated every $C = 10$ episodes. Training occurs over the duration of 2000 episodes, with each episode containing 500 periods.

While we explore the seemingly restrictive setting in which $k = 1$ and firms base their pricing decisions solely on the prices in the preceding period, Calvano et al. (2020) provides an argument in which even when $k = 1$, firms are able to effectively replicate rich strategies that would require an infinitely long memory. Consequently, they find that the standard Q-learning algorithm is indeed able to learn punishment strategies that have finite but long duration but eventually return to collusive prices. With deep neural networks, we find that firms do not learn such strategies due to the nature of the training process: the weights of the network are updated using a random sample of previously stored data. By selecting a random subset

Electronic copy available at: https://ssrn.com/abstract=4489688

of experiences to compute the loss function, which is necessary to train the network and approximate the Q-function accurately, the ability to lock onto supra-competitive focal points as a result of correlated successive state-action pairs is compromised.

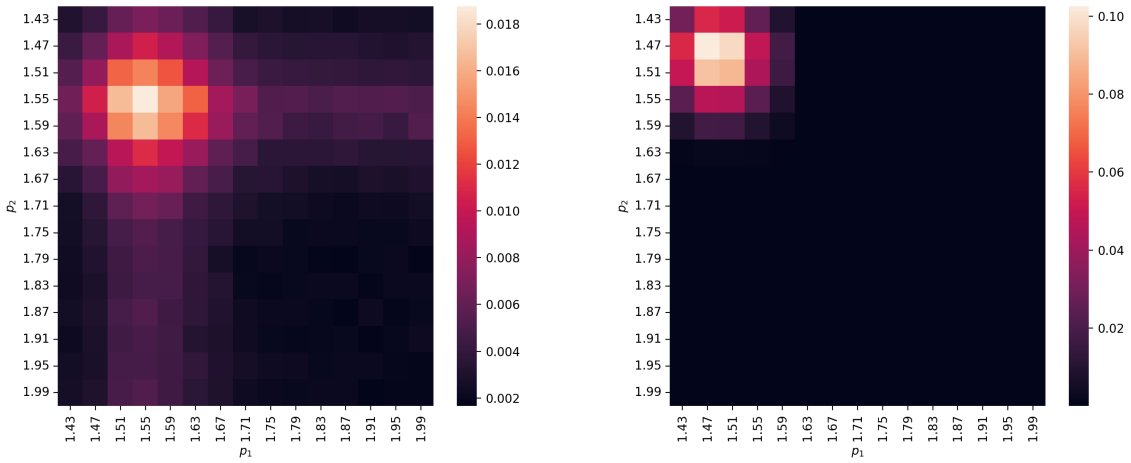

Figure 5.1: Distribution of states visited in the first (left panel) and last (right panel) 100,000 periods

In Figure 5.1, we present the distribution of states visited in the first and last 100,000 periods of the training process. We find that long-run market outcomes largely concentrate around the Nash-Bertrand equilibrium of the stage game. As the training commences, prices will be uniformly distributed across all possible pairs. As the learning progresses and the degree of exploration decreases, prices begin to decrease and cluster closer to the equilibrium, even in the first 100,000 periods of training, displayed in the left panel of the figure. In the right panel, we see that prices concentrate around the equilibrium in the long run. Even though only around one quarter of the prices set are equilibrium prices as presented in Figure 5.2, all prices concentrate close to the equilibrium, as reported in Table 5.1.

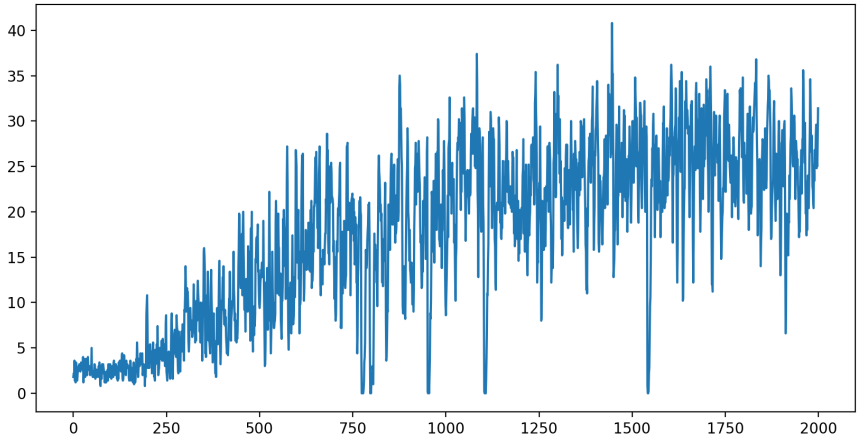

Figure 5.2: Percentage of prices in each episode that are Bertrand-Nash equilibrium of the stage game

Electronic copy available at: https://ssrn.com/abstract=4489688

Table 5.1: Distribution of prices by firm in the last 100,000 periods

Price	1.43	1.47	1.51	1.55	1.59	1.63
Percentage (Firm 1)	17.2%	32.3%	29.0%	14.9%	5.9%	0.7%
Percentage (Firm 2)	17.0%	31.6%	30.3%	15.3%	5.6%	0.6%

Furthermore, we note that within each episode, prices may fluctuate from period to period due to the ongoing training process: each period, a randomly sampled batch is used in the gradient step to update parameters, which changes the Q-values at each state-action pair. The price fluctuation in the long run can be interpreted as fluctuation around a local minimum once the function nears convergence, which may disappear if the learning rate is allowed to decline over the course of training. Nevertheless, the average and most frequently set prices in each episode both take on values close to equilibrium as shown in Figure 5.3. The figure also shows that as the networks are trained and the exploration becomes less extensive, the maximum price in the episode begins to decrease after 500 episodes.

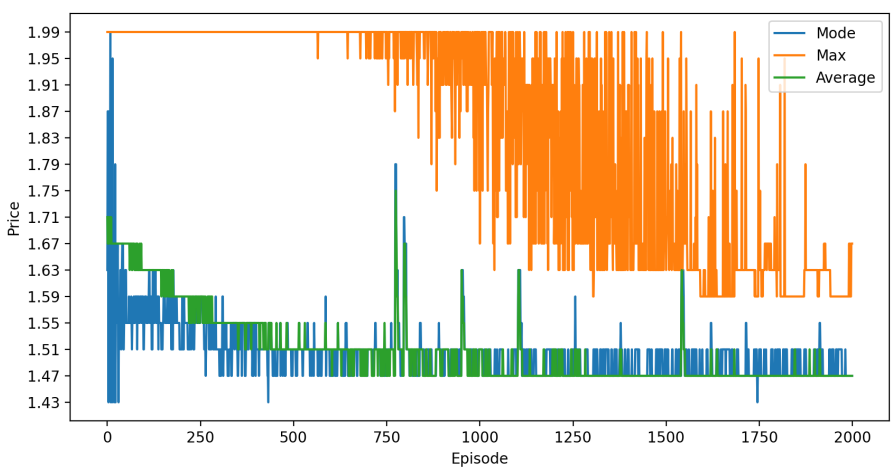

Figure 5.3: The mode, maximum, and average price in each episode

Lastly, we compute the average profit gain Δ in each episode, where $\Delta = \frac{1}{T}\sum_{t=1}^{T}\Delta_t$ and

$$\Delta_t \equiv \frac{\pi_t - \pi^N}{\pi^M - \pi^N},$$

where π^N is the firm profit under the Nash-Bertrand equilibrium in the stage game, π^M is the monopoly or globally maximum profit attainable in the stage game, and π_t is realized profit at period t of the episode. Consequently, $\Delta_t = 0$ when the equilibrium is played in t and $\Delta_t = 1$ when the firms maximize joint profits. This economic indicator is plotted in Figure 5.4. As firms have limited knowledge of the environment and exploration is extensive, Δ is high taking on values close to 0.5. However, as the Q-networks are trained and exploration begins to taper, the profit gain decreases to near equilibrium levels, demonstrating that

Electronic copy available at: https://ssrn.com/abstract=4489688

our formulated deep Q-learning algorithm is unable to coordinate in achieving supra-competitive prices. Similar profit gain dynamics occurs for both firms.

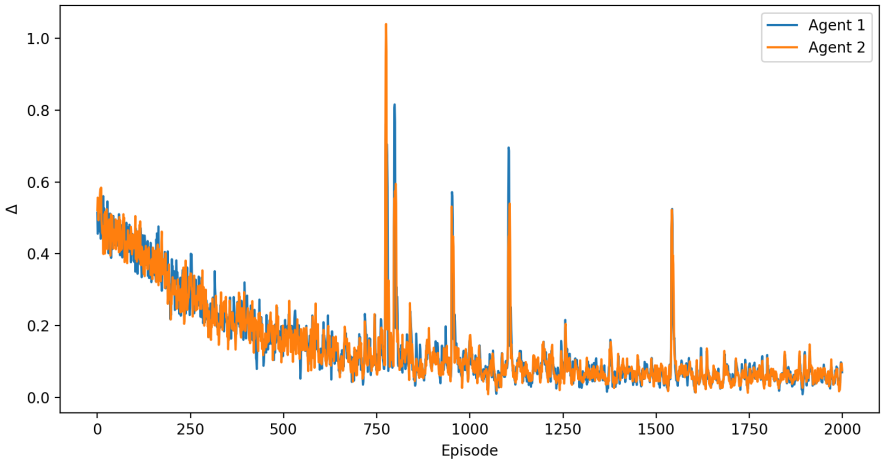

Figure 5.4: Average profit gain in each episode Δ

As a robustness check, we experiment with a wide range of values of characteristic parameters such as α and β, different types of initialization of weights in the networks, as well as the sizes of the replay buffer and mini-batch. Under all cases in which the algorithm is able to stably learn and converge within 100,000 periods of training, the long run market outcomes are close to the Nash-Bertrand equilibrium. Our results are in contrast with Calvano et al. (2020) in which algorithms are capable of generating collusive outcomes and learning rich reward-punishment strategies. The reasoning is as follows. Even in multi-agent environments in which firms adopt tabular Q-learning algorithms, convergence is not guaranteed absent strong assumptions that are not satisfied in our setting as the policies of competitors change over time as a result of learning and experimentation. However, this non-stationarity that hinders learning is partially alleviated by the strong temporal correlations that exist in tabular Q-learning: when an experience is used to update a single cell corresponding to the respective state-action pair cell in the Q-matrix right after it occurs and the successive values of such state-action pairs are temporally correlated, firms more easily discover and lock on to "focal points" at which they learn to have high profits when played, as in the cases where firms both charge supra-competitive prices. When successive state-actions do not vary drastically, the non-stationarity in the learning process is weakened and collusion is sustained. In our deep Q-learning formulation, training is done through experience replay, in which experiences are not immediately used to update the Q-networks as they occur and are instead randomly sampled in later periods which only exacerbates the non-stationarity that precludes the firms from coordinating. Thus the firms are led to play outcomes that are close to equilibrium without learning any reward-punishment schemes. In the next section, we examine whether recurrent neural networks, a class of neural networks for modeling sequential data, are capable of learning grim trigger strategies in order to coordinate their behavior and produce supra-competitive outcomes.

Electronic copy available at: https://ssrn.com/abstract=4489688

6 Deep Recurrent Q-Networks

Recurrent neural networks (RNNs) with are a class of artificial neural networks adapted to model temporal dynamic behavior in sequential data. RNNs are the de facto standard for recognizing and predicting complex patterns in applications including but not limited to including speech recognition, natural language processing, time series prediction, protein homology detection, and video classification. The functional form of the RNN recurrence relation takes the form of $h_t = f_\theta(h_{t-1}, x_t)$, where θ are the weights of the RNN. RNNs, at each period t, takes some input x_t in order to update an internal hidden state h_t which will be fed back into the RNN during the next period at which x_{t+1} is to be inputted and produce an output. The state x_t represents the current state of the system and the hidden state h_t represents the memory of what has occurred previously. The RNN outputs the Q-values corresponding to each feasible action. Such a flexible and rich architecture can potentially allow firms to learn reward-punishment strategies that enable them to collude. We operationalize two types of RNN architectures, the vanilla RNN architecture and the long short-term memory architecture to be detailed in the next subsection, and train them in an fashion analogous to Algorithm 1 in the economic environment specified in Section 3.1.

6.1 Recurrent Neural Network

The most basic recurrent neural network takes values in the state space as inputs and outputs the Q-values for the m possible actions at each time period t. The network consists of a hidden state vector $\boldsymbol{h}_t$ of size h which can be interpreted as the sufficient statistic or memory of the of states of previous periods in a given sequence. The hidden state evolves according to the Markov process:

$$\boldsymbol{h}_t = \mathbf{tanh}\left(\boldsymbol{W}\boldsymbol{x}_t + \boldsymbol{W}^h\boldsymbol{h}_{t-1} + \boldsymbol{b}\right),$$

where $\boldsymbol{x}_t \in \mathbb{R}^{nk}$ denotes the current input vector and $\boldsymbol{b} \in \mathbb{R}^h, \boldsymbol{W} \in \mathbb{R}^{h\times nk}, \boldsymbol{W}^h \in \mathbb{R}^{h\times h}$ are respectively the biases, input weights and recurrent weights. $\mathbf{tanh}(\cdot)$ is the hyperbolic tangent function $\tanh(x) \equiv \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$ applied element-wise. The output $\boldsymbol{h}_t$ in each period t is decoded by applying a linear transformation to obtain vector of size m corresponding to the Q-values associated with the m actions.

6.2 Long Short Term Memory

The *long short-term memory* (LSTM) network first introduced by Hochreiter & Schmidhuber (1997) is a recurrent neural network that takes values in the state space as inputs and outputs the Q-values for the m possible actions at each time period. The network consists of an output state $\boldsymbol{h}_t \in (0,1)^h$ that represents the current, "short-term" state, whereas $\boldsymbol{s}_t$ is a cell state that retains the "long-term memory". These state variables evolve according to the Markov process:

$$\boldsymbol{h}_t = \underbrace{\mathbf{tanh}(\boldsymbol{s}_t)}_{\text{output}} \odot \underbrace{\boldsymbol{\sigma}\left(\boldsymbol{b}^o + \boldsymbol{U}^o\boldsymbol{x}_t + \boldsymbol{W}^o\boldsymbol{h}_{t-1}\right)}_{\text{output gate}}, \quad \text{and}$$

Electronic copy available at: https://ssrn.com/abstract=4489688

$$\boldsymbol{s}_t = \underbrace{\mathbf{tanh}\left(\boldsymbol{b}^s + \boldsymbol{U}^s \boldsymbol{x}_t + \boldsymbol{W}^s \boldsymbol{h}_{t-1}\right)}_{\text{input}} \odot \underbrace{\boldsymbol{\sigma}\left(\boldsymbol{b}^g + \boldsymbol{U}^g \boldsymbol{x}_t + \boldsymbol{W}^g \boldsymbol{h}_{t-1}\right)}_{\text{input gate}} + \boldsymbol{s}_{t-1} \odot \underbrace{\boldsymbol{\sigma}\left(\boldsymbol{b}^f + \boldsymbol{U}^f \boldsymbol{x}_t + \boldsymbol{W}^f \boldsymbol{h}_{t-1}\right)}_{\text{forget gate}},$$

where $\boldsymbol{x}_t \in \mathbb{R}^{nk}$ denotes the current input vector. $\boldsymbol{b}^l \in \mathbb{R}^h, \boldsymbol{U}^l \in \mathbb{R}^{h \times nk}, \boldsymbol{W}^l \in \mathbb{R}^{h \times h}$ are respectively the biases, input weights and recurrent weights for gates $l \in \{f, g, o, s\}$, $\boldsymbol{\sigma}(\cdot)$ is the sigmoid function $\sigma(x) = \frac{1}{1+\exp(-x)}$ applied element-wise, i.e.

$$l_t^{(i)} = \sigma\left(b^{l(i)} + \sum_j U^{l(i,j)} x_t^{(j)} + \sum_j W^{l(i,j)} h_{t-1}^{(j)}\right) \text{ for } l \in \{f, g, o, s\},$$

and $\mathbf{tanh}(\cdot)$ is the hyperbolic tangent function applied element-wise. Lastly, $\odot$ denotes the Hadamard (element-wise) product. The output in each period t is decoded by applying a linear transformation to obtain vector of size m corresponding to the Q-values associated with the m actions. Note that here in our notation, we suppress the indices of the firms and instead use index i to reference the i-th component.

Relative to the vanilla RNN architecture, the advantage of adopting LSTM is two-fold. First, it is more expressive in terms of the ability to regulate the flow of information passed to subsequent periods. Second, adoption of LSTM mitigates the vanishing gradient problem encountered by traditional RNNs. Neural networks are trained via backpropogation, in which the gradient of the loss function is computed with respect to the weights of the feedforward network by application of the chain rule as in Rumelhart et al. (1986). However, in a network with multiple layers, when small derivatives are multiplied together, the gradient decreases exponentially thereby making it difficult to update the parameters in the direction that would improve the cost function and rendering learning unstable.

6.3 Deep Recurrent Q-Learning

The training process is detailed in Algorithm 2. To a multi-agent environment, we adapt the algorithm Hausknecht & Stone (2015), which is the only work we are aware of that investigates deep recurrent Q-learning. The paper drastically improves the performance of the deep Q-learning algorithm in solving partially-observable Markov decision processes by introducing memory into the model, rendering it capable of remembering information through time to inform optimal decisions while only taking in a single state at a given decision period. They show that deep recurrent Q-networks (DRQNs) perform better than DQN under situations in which the state is not fully observable when the networks are trained with full observations and evaluated with partial observations.

With regards to our application, there is a possibility that sequence based neural network modeling can help facilitate algorithmic collusion and enable firms to learn grim trigger strategies more easily. This is in part because the hidden states that are passed to subsequent periods serve as an expressive sufficient statistic of everything that has happened so far, perhaps including whether or not the other firm has deviated, how much the other firm has deviated by, or how many period an ongoing punishment has lasted for. In modern applications of RNNs, complex features that would otherwise require high dimensional input information are learned in a black box nature by the algorithms without any prior feature selection or

Electronic copy available at: https://ssrn.com/abstract=4489688

engineering. While there are no theoretical guarantees that our algorithm would converge, we verify this empirically by experimentation. We report these results in the subsequent subsection.

Algorithm 2: Multi-agent deep recurrent Q-learning with random experience replay

```
For each agent i = 1, ..., n:
Initialize a replay memory D^i with capacity N;
Initialize a Q-network Q^i with random weights θ^i;
Initialize a target Q-network Q̂^i with random weights θ̂^i;
Initialize a random state s_1;
for episode = 1,..., M do
    for period t = 1,...,T do
        For each agent i, with probability 1 − ε_t select a greedy action
        a_t^i = argmax_a Q^i(s_t, a; θ^i); otherwise select a random action a_t^i
        Execute actions (a_t^1, ..., a_t^n) together and each agent observes all rewards r_{i,t}
        Set state s_{t+1} according to actions taken
    end
    Store episode history {s_1, (a_1^1, ...a_1^n), s_2, (a_2^1, ..., a_2^n), ..., s_T, (a_T^1, ..., a_T^n)} in every D^i
    Sample random mini-batch of episode histories independently from D^i
    Initialize h^(0), s^(0)
    for period t = 1,...,T do
        For each agent i, perform a gradient descent step on individual loss function with respect to
          weights θ^i, i.e. θ ← θ − α∇_θ loss
    end
    Every C episodes, update target network for each agent, i.e. θ̂^i ← θ^i
end
```

6.4 Results

We train the DRQNs for a total of 100,000 episodes, with each episode containing only 10 periods to reduce the problem of vanishing and exploding gradients which hinders the learning process when the sequence is sufficiently long. As a result, memory in our model lasts for 10 periods before it is reset in the next episode. We examine both RNN and LSTM networks and the market outcomes induced by the learning process under two scenarios. The first scenario is when one firm trains a RNN against a stationary opponent. That is, we ask whether an RNN is able to learn optimal best response strategies in a situation where the rival firm implements a strategy that is fixed in each episode. The other scenario is one in which the recurrent Q-network of a firm is trained against another firm that is simultaneously learning using the same network. In Figure 6.1, we show two examples of the pricing response of a firm after its DRQN is trained in settings where its rival adopts a fixed strategy each period.

Electronic copy available at: https://ssrn.com/abstract=4489688

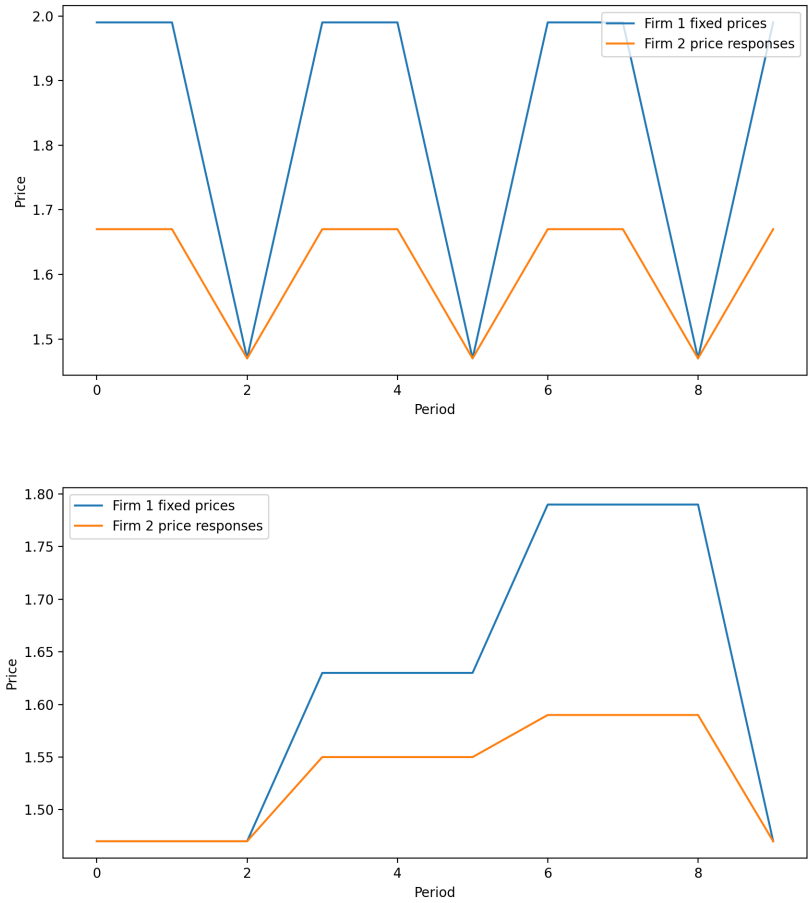

Figure 6.1: AI trained price responses of firm 1 to a fixed strategy of firm 2. The top panel presents the case in which firm 2 sets prices to 1.99 for two periods followed by 1.47 for one period and repeats the process until the end of the episode. The bottom panel presents the case in which firm 2 sets 1.47 for three periods, followed by 1.63 for the next three, followed by 1.79 for the next three, and then repeating this cycle.

In the top panel, firm 1 sets prices to 1.99 for two periods followed by 1.47 for one period and repeats the process until the end of the episode whereas in the bottom panel, firm 1 sets 1.47 for three periods, followed by 1.63 for the next three, followed by 1.79 for the next three, and then repeating this cycle. In both cases, firm 2 is able to play best responses to the prices set by firm 1 in 100% of the periods following training. In the former case, firm 2 is capable of anticipating, that after firm 1 sets prices to 1.99 in two consecutive periods, the firm will cut its price to 1.47 in the subsequent period. Consequently, given only information about the price firm 1 set in the preceding period, firm 2 plays the best response of 1.67 during all periods in which firm 1 plays 1.99 and the best response of 1.47 whenever firm 1 plays 1.47. In the latter case, firm 2 knows that after firm 1 plays 1.47 for three periods, firm 1 will increase and set prices to 1.63 for the next three periods, and so on until the cycle resets. When firm 1 exercises a stationary strategy, firm 2 can be trained to anticipate how firm 1 will price in the next period and accordingly play a best response. Even

Electronic copy available at: https://ssrn.com/abstract=4489688

when the only input to the network in each period is the state s_t, the price of firm 1 in the preceding period, vanilla RNNs are able to stably learn how to best respond 100% of the periods after training is complete. This finding suggests that perhaps antitrust authorities should be concerned in which a single firm adopts fixed pricing that set extraordinarily high prices and its rival firm learns to best respond to those prices via deep recurrent neural networks, leading to supra-competitive prices overall.

In the second scenario, in which two firms train DRQNs simultaneously, the propensity of either firm to set supra-competitive prices is low. Over the course of the 100,000 episodes of training, while the number of unique prices set by the firms decreases to between two and four as exploration becomes less extensive over time, market outcomes tend towards equilibrium level as shown in Figure 6.2 for RNNs and Figure 6.3 for LSTMs. In the former case, the profit gain Δ tends towards to 0, and the variance of Δ decreases over time. We observe similar patterns for the case of LSTMs even though there remains considerable variance towards the end of training. In both cases, except by fluke, a vast majority of the prices set are not best responses and firms do not learn any grim trigger or reward-punishment strategies despite the flexibility of the RNN model. This suggests that firms are not only unable to coordinate, they are unable to even anticipate the future moves of their rivals, even if they are constant throughout the episode, and play best responses. This lack of convergence to any strategy that resembles best responding is a result of non-stationarity in training when both firms are experimenting and learning new policies at the same time. It remains unclear under which formulation, if any at all, would firms be able to simultaneously train sequence based models to collude or even best respond to each other's pricing strategies.

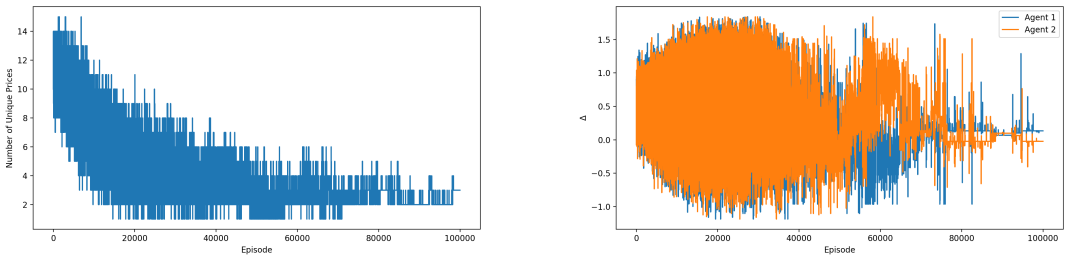

Figure 6.2: Number of unique prices (left) and average profit gain (right) Δ in each episode for RNNs

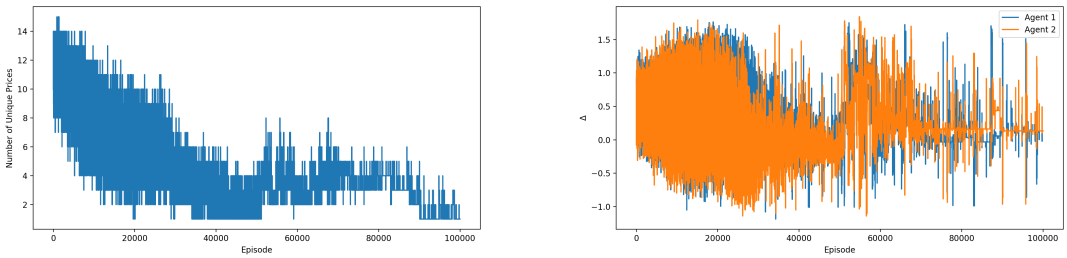

Figure 6.3: Number of unique prices (left) and average profit gain (right) Δ in each episode for LSTMs

Electronic copy available at: https://ssrn.com/abstract=4489688

7 Conclusion

We examined the function approximation analog of works akin to Calvano et al. (2020) and Asker et al. (2022) in which AI algorithms consistently learn to charge supra-competitive prices without communication. In this stream of works, the values associated with a particular state-action pair are updated directly in a tabular manner. However, the memory required by tabular representation of such state-action values in Q-learning increases combinatorially in the number of possible states and actions. As a result, when $|\theta| \ll |S \times A|$ holds true, as it likely does in high-dimensional real-world applications where tabular Q-value representations becomes intractable to learn, firms would naturally rely on deep neural networks to represent their policies. Deep neural networks approximate underlying relationships between state-action pairs and rewards as well as generalize knowledge collected from previous experiences to unseen states of the environment.

Hence, we present an alternative view of pricing through AI relative to Calvano et al. (2020) and Asker et al. (2022). We examined algorithms in which the parameters θ of a function-approximating neural network are updated according to experience replay instead of individual entries in a tabular Q-matrix. After training concludes, the outcomes are close to the equilibrium of the stage game, a finding in contrast with previous works. This result is attributed to the nature by which the neural networks are trained. In order to ensure stable training, data is stored in a replay buffer to be sample when computing the gradient in future periods, breaking the temporal correlation between successive states and actions that promoted firms to learn supra-competitive prices in the tabular case.

We further introduce memory into the deep neural network model by means of adopting a recurrent neural network. While such sequence based models perform well in stationary environments in which the rival firm exercises a fixed pricing strategy and are able to best respond during 100% of the periods following convergence, the ability to learn any meaningful strategy is significantly compromised when multiple firms are simultaneously learning. Our negative results suggest that it is far more difficult than postulated in the literature for firms to learn to achieve tacit collusion through AI algorithms, even when the economic primitives of the problem are stationary. Surely, under stochastic environments and more complex state-action spaces, collusion is far more unlikely. Algorithmic collusion may not be an immediate concern to antitrust authorities nor a threat to competitive markets. There remains a long way to go before artificial intelligence and reinforcement learning can be effectively applied in reality, owing to instability of the training processes induced by the non-stationarity nature of real-world problems.

Electronic copy available at: https://ssrn.com/abstract=4489688

References

Asker, J., C. Fershtman, and A. Pakes (2022): "Artificial Intelligence and Pricing: The Impact of Algorithm Design," *AEA Papers and Proceedings, forthcoming*.

Assad, S., R. Clark, D. Ershov, and L. Xu (2020): "Algorithmic Pricing and Competition: Empirical Evidence from the German Retail Gasoline Market," *working paper*.

Brown, Z. Y. and A. MacKay (2021): "Competition in Pricing Algorithms," *American Economic Journal: Microeconomics, forthcoming*.

Calvano, E., G. Calzolari, V. Denicolò, and S. Pastorello (2020): "Artificial Intelligence, Algorithmic Pricing, and Collusion," *American Economic Review*, 110.

Chen, L., A. Mislove, and C. Wilson (2016): "An Empirical Analysis of Algorithmic Pricing on Amazon Marketplace," *roceedings of the 25th International Conference on World Wide Web*, 1339–1349.

Chen, N. and H.-T. Tsai (2021): "Steering via Algorithmic Recommendations," *working paper*.

Fan, J., Z. Wang, Y. Xie, and Z. Yang (2020): "A Theoretical Analysis of Deep Q-Learning," *Proceedings of the 2nd Conference on Learning for Dynamics and Control*, 120, 486–489.

Gomez-Uribe, C. A. and N. Hunt (2016): "The Netflix Recommender System: Algorithms, Business Value, and Innovation," *ACM Transactions on Management Information Systems*, 6, 1–19.

Hansen, K. T., K. Misra, and M. M. Pai (2021): "Frontiers: Algorithmic Collusion: Supra-competitive Prices via Independent Algorithms," *Marketing Science*, 40, 1–191.

Harrington Jr., J. E. (2018): "Developing Competition Law for Collusion by Autonomous Artificial Agents," *Journal of Competition Law and Economics*, 14, 331–363.

——— (2021): "The Effect of Outsourcing Pricing Algorithms on Market Competition," *working paper*.

Hausknecht, M. and P. Stone (2015): "Deep Recurrent Q-Learning for Partially Observable MDPs," *Association for the Advancement of Artificial Intelligence*.

Hochreiter, S. and J. Schmidhuber (1997): "Long Short-Term Memory," *Neural Computation*, 1735–1780.

Klein, T. (2021): "Autonomous algorithmic collusion: Q-learning under sequential pricing," *The RAND Journal of Economics*, 52, 538–558.

Leisten, M. (2022): "Algorithmic Competition, with Humans," *working paper*.

Miklós-Thal, J. and C. Tucker (2019): "Collusion by Algorithm: Does Better Demand Prediction Facilitate Coordination Between Sellers?" *Management Science*, 65, 1552–1561.

Electronic copy available at: https://ssrn.com/abstract=4489688

Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis (2015): "Human-level control through deep reinforcement learning," *Nature*, 518, 529–533.

O'Connor, J. and N. E. Wilson (2021): "Reduced demand uncertainty and the sustainability of collusion: How AI could affect competition," *Information Economics and Policy*, 54.

Salcedo, B. (2015): "Pricing Algorithms and Tacit Collusion," *working paper*.

Vinyals, O., I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J. P. Agapiou, M. Jaderberg, A. S. Vezhnevets, R. Leblond, T. Pohlen, V. Dalibard, D. Budden, Y. Sulsky, J. Molloy, T. L. Paine, C. Gulcehre, Z. Wang, T. Pfaff, Y. Wu, R. Ring, D. Yogatama, D. Wünsch, K. McKinney, O. Smith, T. Schaul, T. Lillicrap, K. Kavukcuoglu, D. Hassabis, C. Apps, and D. Silver (2019): "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, 575, 350–354.

Watkins, C. (1989): "Learning from Delayed Rewards," *Ph.D. thesis*.

Watkins, C. J. C. H. and P. Dayan (1992): "Artificial Intelligence and Pricing: The Impact of Algorithm Design," *Machine Learning*, 8, 279–292.

Zhang, K., Z. Yang, and T. Başar (2021): "Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms," *Handbook of Reinforcement Learning and Control*, 321–384.

Electronic copy available at: https://ssrn.com/abstract=4489688